

STEM Course Prep
Class Activities
Chapter 5

Slides 1-9

1. Use a for loop to count to a number suggested by the user.

```
num=input('Enter a number to count to: ');  
  
for i=1:num  
    fprintf('count= %d \n',i)  
end
```

2. Use a for loop to add 5 to a number 10 times.

```
sum=0;  
for i=1:10  
    sum=sum+5;  
    fprintf('sum = %.2f when i = %d\n',sum,i)  
end
```

3. You have a vector of numbers and you want to multiply each of the numbers by 2. Use a for loop to do this.

```
Vector = [9 5 3 7 8];  
for Index = 1:length(Vector)  
    Vector(Index) = Vector(Index)*2;  
end  
disp(Vector)
```

4. Using the kinematic equations we will plot the trajectory of a projectile for angles ranging from 5 degrees to 85 degrees in 10 degree increments. All plots will be on the same figure. Add text that states the angle of each trajectory.

```
vo = 150; % initial velocity, m/s  
a = -9.81; % acceleration, m/s^2  
  
hold on % put multiple plots on one figure  
  
% loop through each angle and plot the trajectory  
  
for angle=5:10:85  
    vox = vo*cosd( angle ); % initial x-velocity  
    voy = vo*sind( angle ); % initial y-velocity  
    tf = -2*voy/a; % time of flight  
    t = linspace(0,tf); % create time points for the trajectory plot  
    x = vox * t; % x-position at each point in time  
    y = voy * t + 0.5*a*t.^2; % y-position at each point in time  
    Continued on next page...
```

```

plot( x, y)    % plot the trajectory that corresponds to the angle.

% Find the point where y is at a max value. Put the corresponding
% angle on the plot at this location.

text(max(x)/2,max(y),num2str(angle))
end

grid on
axis tight
xlabel('x (m)')
ylabel('y (m)')
title('Projectile Trajectories')

```

Slides 10-13

5. Calculate the velocity for time values of 1 , 2, 3, 4, 5. Initial velocity is 10 and acceleration is 2. Store the numbers in a vector. Do it once without preallocation and once with preallocation. Use the tic/toc to time the process.

No preallocation:

```

clear all
tic
for t=1:5
    vel(t)=10+2*t;
end
disp(vel)
toc

```

With preallocation:

```

clear all
tic
vel=zeros(5,1);
for t=1:5
    vel(t)=10+2*t;
end
disp(vel)
toc

```

Slides 14-16

6. Use the subplot function to create a 2x3 matrix of plots .

```

for i=1:20
    x(i) = i-10;
    squared(i) = x(i) ^ 2;
    cube(i) = x(i) ^ 3;
    linear(i) = x(i);
    log_of(i) = log(x(i));
    sqrt_of(i) = sqrt(x(i));
end

subplot(2,3,1)
plot(x,squared)
title('square')
grid on

subplot(2,3,3)
plot(x,cube)
title('cube')

subplot(2,3,4)
plot(x,sqrt_of, 'ro')
title('sqrt')
axis tight

subplot(2,3,5)
plot(x,linear )
title('linear')
xlabel('x')
ylabel('x')

subplot(2,3,6)
plot(x,log_of)
title('log')

```

7. Given an initial position and acceleration, calculate and plot the final position after 10 seconds. Use 100 linearly spaced times from 0 to 10. Use subplot to generate a 2x2 grid of plots.

```

initial_position=100;
initial_vel=[150 350 400 475];
accel=10;
t=linspace(0,10,100);
for i=1:4
    final_position=initial_position+initial_vel(i)*t+.5*accel*t.^2;
    subplot(2,2,i)
    plot(t,final_position)
    xlabel('t')
    ylabel('position')
end

```

Slides 17-19

8. Write a script that will output all the elements of a variable-size matrix one at a time.

```

m=input('Enter the number of rows');
n=input('Enter the number of columns');
matrix=randi([1 10],m,n)
%[m n] = size(matrix); not necessary but size could be used
for Row = 1:m
    for Col = 1:n
        fprintf('Element (%d,%d) = %d.\n', Row, Col, matrix(Row, Col))
    end
end

```

9. Find the transpose of matrix A using nested for loops.

```

A=[2 1 -1;3 1 2;0 -2 -3];
for i =1:3
    for j=1:3
        A_transpose(j,i)=A(i,j);
    end
end

```

10. Download grades2.txt from the website. Use nested for loops to search for A and B exam grades. Print the ID number of the student and state what exam the A or B grade was on.

```

load grades2.txt

for i=1:10
    for j=3:5
        if grades2(i,j)>=90
            fprintf('Student %d made an A on exam %d\n',grades2(i,1),j-2)
        elseif grades2(i,j)>=80 & grades2(i,j)<90
            fprintf('Student %d made a B on exam %d\n',grades2(i,1),j-2)
        end
    end
end

```

11. The following equation is the standard form for the first order response of a system.

$$c(t) = R - Re^{-at}$$

where R =input value and a =reciprocal of the time constant. Plot the time response of a first order system for two different values of unit step inputs and various values of the reciprocal of the time response, a . Label each line with the value of a .

```

clear all

a = [0.1 0.2 0.5 .75 1];           % set the rate constants

hold on

% loop through each rate constant and input value. Plot the first order response.

for R=1:2
    for i=1:length(a)
        t = linspace(0,4/a(i));    % set the time over which we will calculate c
        c = R-R*exp(-a(i)*t);
        if R==1
            plot(t,c, 'r')         % plot the c vs time line
        else
            plot(t,c)
        end
        text(max(t),max(c),num2str(a(i)))
    end
end
end

```

Slides 20-26

12. You are designing a beam that will support a load at one end. You want to ensure that you don't exceed the maximum moment allowed which is 150 ft-lbs. In order to test your beam you create a script that calculates the moment as weight is added to the end of the beam in 5 lb increments. Once the maximum weight is determined, the script will output the number of weights added and the max possible weight, as well as the max moment reached. The length of the beam is 2 feet. Recall moment is equal to the force times the perpendicular distance.

```

weight=0; %initialize weight
momentA=0; %initialize moment about A
d=2; %moment arm distance
i=0; %initialize weight counter
hold on
while momentA<150
    weight = weight+5; %add another weight if max moment not reached
    momentA=weight*d; %calculate the moment
    plot(weight,momentA, 'ro') %plot
    i=i+1; %increment weight counter
end
xlabel('weight')
ylabel('moment about A')
fprintf('The number of possible weights is %d \n',i)
fprintf('The max weight is %d \n',weight)
fprintf('The moment reached at the max weight is %d \n',momentA)

```

13. You have invested money in a bank account that compounds annually. The interest rate is 10%. You want to know how long it takes for the account balance to double. You also want to know the balance of the account each year until the account doubles. Use an initial balance of \$1000.

```

init_bal=1000; %initial balance
rate=0.1; %interest rate
bal=init_bal;
year =0; %initialize the year
fprintf('Year      Balance\n')

while bal < 2*init_bal %check to see if the current balance is double the
initial balance
    bal=bal+rate*bal; %increase the balance amount to account for compound
interest
    year=year+1; %update the year
    fprintf(' %d      %.2f\n',year, bal)%display the year and current
balance for that year
end

```

14. Going back to the projectile problem we have looked at before.... This time we will assume we don't know the time of flight. We want to plot the trajectory as long as the y-component of position is greater than or equal to zero. We will use a while loop.

```

clear all
ang=input('Enter the launch angle in degrees: \n');
dt=0.1; %time increment
g=9.81; %accel due to gravity
v_o=60; %initial velocity
y=0; %initialize temp variable for y-component of position
i=1; %initialize counter for position arrays
t=0; %initialize time

while y>=0
    y(i)=v_o*sind(ang)*t- .5*g*t^2;
    x(i)=v_o*cosd(ang)*t;
    t=t+dt;
    i=i+1;
end
plot(x,y, 'ro')
grid on
xlabel('x (m)')
ylabel('y (m)')
title('Projectile Trajectory')

```

15. Create a script that will allow you to calculate how many months it will take to pay off credit card debt. Inputs: APR, initial balance. Plot the balance and provide output stating how many years it will take to pay off the balance and the total interest charges. Note: APR must be entered as a percentage.

```

clear all

i=1;
tot_fin_charges=0;
APR=input('Enter the APR for your credit card: ');
bn(i)=input('Enter the initial balance in dollars: ');
r=(APR/100)/12; %monthly rate

while bn>0
    pn(i) = max((r + 0.01)*bn(i), 10); %minimum payment
    fn(i) = r * bn(i); %interest to add to balance
    bn(i+1) = bn(i) + fn(i) - pn(i); %new balance
    tot_fin_charges=tot_fin_charges+fn(i);
    i=i+1;
end
fprintf('It will take %.2f years to pay off your debt\n',i/12)
fprintf('Total finance charges at %d percent = $%.2f \n',APR,tot_fin_charges)
plot(bn)
grid on
axis tight
ylabel('balance ($)')
xlabel('months')

```

Slides 27-28

16. Generate 10,000 temperatures in Fahrenheit and then convert them to Celsius. Do the conversion in two ways: looping and vectorized code. Time both methods. Plot the results to ensure results are the same.

```

clear all
time=linspace(0,365,1e4);%linearly space 10,000 data points
tempF=-20*cosd(time); %use cos function just to generate shape

tic %set timer 1
for i=1:length(time)
    tempC1=(5/9)*(tempF(i)-32); %convert to degrees F using looping
end
t1=toc; %stop timer 1

tic %start timer 2
tempC2=(5/9)*(tempF-32);%convert to degrees F with vectorized code
t2=toc; %stop timer 2

hold on
plot(time,tempF,'c-','linewidth',3) %linewidth is another way to customize plots
plot(time,tempC1,'b','linewidth',6)
plot(time,tempC2,'m','linewidth',2)
legend('Temp F', 'Temp C looping', 'Temp C vectorized')
xlabel('Time (days)')
ylabel('Temperature')

fprintf('looping took %.8f seconds\n',t1)
fprintf('vectorized code took %.8f seconds\n',t2)

```